

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

ÓrbiKa: Plataforma Web para la Comercialización y Revalorización de Productos Perecibles orientada al Consumo Sostenible en la Provincia de Tacna

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Renzo Fernando Loyola Vilca ***(2021072615)***

Brant Antony Chata Choque ***(2020067577)***

Tacna – Perú

2026

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	BCC	RLV	BCC, RLV	10/10/2026	Versión original del Informe de Métricas del Proyecto.

"ÓrbiKa: Plataforma Web para la Comercialización y Revalorización de Productos Perecibles"

Informe de Métricas del Proyecto

Versión 1.0 — Corte al 10 de octubre de 2026

Índice General

1. INTRODUCCIÓN **4**
2. TABLERO RESUMEN DE MÉTRICAS **6**
3. MÉTRICAS DE DOCUMENTACIÓN Y ESPECIFICACIÓN **7**
A1. Cobertura de especificación de requerimientos funcionales 7
A2. Cobertura de casos de uso narrados 7
A3. Trazabilidad de requerimientos (RF ↔ CU ↔ Objetivo de negocio) 7
A4. Completitud del modelo de vistas 4+1 (SAD) 8
A5. Inserción de prototipos visuales (wireframes) en el SRS 8
4. MÉTRICAS DE CONSTRUCCIÓN DE SOFTWARE (CÓDIGO) **9**
B1. Tamaño del software construido (líneas de código) 9
B2. Cobertura de casos de uso implementados en código 9
B3. Complejidad ciclomática promedio (McCabe) 9
B4. Densidad de comentarios en el código 10
B5. Modularidad y separación de responsabilidades 10
5. MÉTRICAS DE PRUEBAS Y ASEGURAMIENTO DE CALIDAD **11**
C1. Cobertura de casos de uso con plan de pruebas 11
C2. Automatización de pruebas planificadas 11
C3. Ejecución de pruebas y densidad de defectos 11
6. MÉTRICAS DE ATRIBUTOS DE CALIDAD EN OPERACIÓN **12**
D1. Rendimiento bajo carga concurrente 12
D2. Disponibilidad del servicio (uptime) 12
D3. Usabilidad del registro sin capacitación previa 12
D4. Mantenibilidad y costo de cambio 13
7. HALLAZGO: ACTUALIZACIÓN DEL AVANCE DE CONSTRUCCIÓN **14**
8. CONCLUSIONES **15**
¿Qué métricas resultaron más relevantes según el tipo de software? 15
¿Hubo desacuerdos entre grupos? ¿Por qué? 15

1. INTRODUCCIÓN

Este informe mide el estado real del proyecto ÓrbiKa a partir de tres fuentes verificables: los documentos formales entregados (FD01, SRS v2.1, SAD y Catálogo de Pruebas), el código fuente efectivamente construido (backend en Node.js/Express, frontend en React 18 y el esquema SQL en Supabase/PostgreSQL), y el plan de pruebas diseñado para el sistema. No se reportan cifras estimadas: cada métrica indica en su ficha cómo fue calculada y sobre qué artefacto.

Las métricas se agrupan en cuatro categorías: (A) Documentación y Especificación, (B) Construcción de Software, (C) Pruebas y Aseguramiento de Calidad, y (D) Atributos de Calidad en Operación. Esta última categoría corresponde a los ocho escenarios de calidad definidos en el SAD (rendimiento, disponibilidad, usabilidad y mantenibilidad); como el sistema aún no está desplegado en un entorno productivo con usuarios reales, la mayoría de sus métricas se reportan como No aplica (N/A), con la justificación correspondiente — tal como exige medir honestamente el estado actual de un proyecto en construcción, en lugar de proyectar cifras no verificadas.

Cada ficha de métrica sigue el formato: Definición (qué mide), Justificación (por qué importa para este tipo de software), Cómo se mide (fuente y método de cálculo) y Resultado (valor obtenido, con el estado: cumple la meta, hallazgo parcial, o no aplica).

2. TABLERO RESUMEN DE MÉTRICAS

Vista consolidada de las 17 métricas evaluadas. El detalle de cada una —definición, justificación y método de cálculo— se desarrolla en las secciones 3 a 6.

ID	Métrica	Resultado	Estado
A1	Cobertura de RF documentados	21 / 21 = 100%	✓
A2	Cobertura de CU narrados	21 / 21 = 100%	✓
A3	Trazabilidad RF↔CU↔Objetivo	21 / 21 filas = 100%	✓
A4	Completitud del modelo de vistas 4+1	5 / 5 vistas = 100%	✓
A5	Inserción de prototipos visuales en el SRS	0 / 21 wireframes = 0%	⚠
B1	Tamaño del software construido	1,354 LOC en 30 archivos	—
B2	Cobertura de CU implementados en código	11 / 21 = 52.4% (back); 9 / 21 = 42.9% (completos)	⚠
B3	Complejidad ciclomática promedio	4.21 (McCabe); máx. 16	✓
B4	Densidad de comentarios en el código	81 / 1,354 = 6.0%	⚠
B5	Modularidad / arquitectura en capas	30 / 30 archivos = 100%	✓
C1	Cobertura de CU con plan de pruebas	21 / 21 = 100%	✓
C2	Automatización de pruebas planificadas	21 / 63 = 33.3%	⚠
C3	Ejecución de pruebas y densidad de defectos	0 / 63 ejecutadas	N/A
D1	Rendimiento bajo carga concurrente	Sin entorno productivo	N/A
D2	Disponibilidad del servicio (uptime)	Sin despliegue activo	N/A
D3	Usabilidad del registro sin capacitación	Sin pruebas de usuario	N/A
D4	Mantenibilidad / costo de cambio	Sin cronometraje real	N/A

Leyenda: cumple la meta u objetivo definido (✓) · hallazgo parcial o por debajo del umbral recomendado (⚠) · N/A no aplica todavía (sistema sin despliegue productivo o sin ejecución formal) · — valor absoluto, sin umbral de aprobación.

3. MÉTRICAS DE DOCUMENTACIÓN Y ESPECIFICACIÓN

Miden qué tan completa y trazable quedó la especificación del sistema antes de construirlo. Para un proyecto académico evaluado por fases, son las primeras métricas disponibles y las que sustentan todas las demás: nada de lo que se mida en el código tiene sentido si no está primero bien especificado.

A1. Cobertura de especificación de requerimientos funcionales

Elemento	Descripción
Definición	Proporción de requerimientos funcionales (RF) identificados en el análisis que cuentan con una descripción formal, módulo asignado y prioridad en el SRS.
Justificación	Un RF sin especificar no puede construirse ni probarse de forma confiable; esta métrica es la base de todas las demás.
Cómo se mide	Conteo directo de filas de la tabla de Requerimientos Funcionales del SRS v2.1 (sección 2.1.1).
Resultado	**21 de 21 RF especificados = 100%**

A2. Cobertura de casos de uso narrados

Elemento	Descripción
Definición	Proporción de casos de uso (CU) identificados que cuentan con narrativa completa en formato Actor/Sistema (flujo principal y alternativas y errores).
Justificación	La narrativa del CU es el contrato funcional que el equipo de construcción implementa; sin ella, el desarrollo se basa en supuestos.
Cómo se mide	Conteo directo de las 21 fichas CU-01 a CU-21 en el SRS, verificando presencia de flujo principal y alternativas.
Resultado	**21 de 21 CU narrados = 100%**

A3. Trazabilidad de requerimientos (RF ↔ CU ↔ Objetivo de negocio)

Elemento	Descripción
Definición	Proporción de RF que están vinculados explícitamente a un caso de uso y a un objetivo de negocio del proyecto.
Justificación	Sin trazabilidad no es posible justificar por qué existe cada línea de código ni medir el impacto de un cambio de alcance.
Cómo se mide	Conteo de filas completas (sin celdas vacías) en la Matriz de Trazabilidad del SRS (sección VI.1).
Resultado	**21 de 21 RF trazados = 100%**

A4. Completitud del modelo de vistas 4+1 (SAD)

Elemento	Descripción
Definición	Proporción de las cinco vistas arquitectónicas del modelo de Kruchten (Casos de Uso, Lógica, Implementación, Procesos, Despliegue) que cuentan con al menos un diagrama en el SAD.
Justificación	Un SAD con vistas incompletas deja decisiones de arquitectura sin sustento visual, lo que dificulta construir y mantener el sistema de forma consistente.
Cómo se mide	Revisión directa de la sección 3 del SAD_ORBIKA.docx: 8 diagramas distribuidos en las 5 vistas (casos de uso, paquetes, secuencia ×2, colaboración, objetos, clases, entidad-relación, arquitectura de paquetes, componentes, despliegue).
Resultado	**5 de 5 vistas documentadas = 100%**

A5. Inserción de prototipos visuales (wireframes) en el SRS

Elemento	Descripción
Definición	Proporción de los 21 prototipos rústicos generados (uno por caso de uso) que ya están insertados dentro del documento SRS entregado.
Justificación	El docente exige explícitamente que cada caso de uso tenga su prototipo visual dentro del documento; sin esa inserción, la evidencia existe pero no es verificable en la entrega formal.
Cómo se mide	Conteo de imágenes en word/media del SRS_ORBIKA.docx (11, todas correspondientes a diagramas UML) frente a los 21 archivos PNG de wireframes generados en la carpeta de trabajo, ninguno de los cuales aparece insertado.
Resultado	**0 de 21 wireframes insertados = 0%**

4. MÉTRICAS DE CONSTRUCCIÓN DE SOFTWARE (CÓDIGO)

Se calculan directamente sobre el código fuente real del proyecto (carpeta orbika-app: backend Node.js/Express, frontend React 18, esquema SQL de Supabase), no sobre estimaciones. El detalle por archivo se obtuvo con un script de conteo de líneas y un analizador de complejidad ciclomática construidos para este informe.

B1. Tamaño del software construido (líneas de código)

Elemento	Descripción
Definición	Cantidad de líneas de código efectivas (excluyendo líneas en blanco y comentarios) en el repositorio del proyecto.
Justificación	Es la métrica de tamaño más básica del producto; sirve de denominador para normalizar otras métricas (densidad de comentarios, densidad de defectos).
Cómo se mide	Script de conteo línea por línea sobre los 30 archivos fuente (.js, .jsx, .sql, .css), clasificando cada línea como código, comentario o blanco.
Resultado	**1,354 LOC de código · 303 LOC backend (9 archivos) · 841 LOC frontend (18 archivos) · 201 LOC SQL (2 archivos) · 1,596 líneas totales**

B2. Cobertura de casos de uso implementados en código

Elemento	Descripción
Definición	Proporción de los 21 CU especificados que ya cuentan con un endpoint de backend funcional, y proporción que además tiene interfaz de usuario conectada (experiencia completa).
Justificación	Es el indicador más directo de avance real de construcción; a diferencia del SRS o el SAD (100% especificados), revela cuánto de lo especificado ya es software que corre.
Cómo se mide	Búsqueda de comentarios de trazabilidad «CU-XX» en el código fuente (rutas, controladores y páginas), contrastado contra la existencia de una página de frontend que consuma ese endpoint.
Resultado	**11 de 21 CU con backend = 52.4% (CU-01,02,03,04,05,06,07,08,13,17,19) · 9 de 21 con experiencia completa = 42.9% (todos los anteriores salvo CU-06 y CU-19, que solo tienen servicio y ruta, sin pantalla)**

B3. Complejidad ciclomática promedio (McCabe)

Elemento	Descripción
Definición	Número promedio de caminos de decisión independientes por función (1 + cantidad de if, else if, for, while, case, catch, &&, ||, ?? y operador ternario).
Justificación	Una complejidad baja (idealmente ≤10) indica funciones más fáciles de entender, probar y modificar sin introducir errores; es especialmente relevante porque el equipo seguirá construyendo los 10 CU restantes sobre esta misma base.
Cómo se mide	Analizador de complejidad por patrones léxicos sobre las 33 funciones detectadas en los 23 archivos .js/.jsx del proyecto (aproximación por conteo de puntos de decisión, no un parser de AST completo).

Elemento	Descripción
Resultado	**Promedio 4.21 · máximo 16 (componente ProductDetail, por la cantidad de renderizados condicionales en JSX) · 2 de 33 funciones (6.1%) superan el umbral de riesgo de CC>10**

B4. Densidad de comentarios en el código

Elemento	Descripción
Definición	Proporción de líneas de comentario respecto de las líneas de código efectivas.
Justificación	Un nivel adecuado de comentarios (orientativamente 10%–20% en proyectos académicos) facilita que un segundo integrante del equipo entienda y extienda el código sin depender solo de quien lo escribió.
Cómo se mide	Cociente entre líneas de comentario y líneas de código del conteo de la métrica B1.
Resultado	**81 / 1,354 = 6.0%, por debajo del rango recomendado de 10%–20%**

B5. Modularidad y separación de responsabilidades

Elemento	Descripción
Definición	Proporción de archivos de código que respetan la arquitectura en capas definida en el SAD (config, middlewares, services, controllers, routes en el backend; pages, components, context, services en el frontend), sin lógica de negocio mezclada en una capa que no le corresponde.
Justificación	Sustenta la meta de mantenibilidad del SAD (escenario «Incorporación de un Nuevo Método de Entrega», <4 h-persona): una arquitectura ordenada es la precondición para que un cambio futuro sea rápido y aislado.
Cómo se mide	Revisión manual de los 30 archivos fuente contra la estructura de carpetas declarada en el SAD (sección 3.3.1, Figura 7).
Resultado	**30 de 30 archivos respetan su capa = 100%, 0 violaciones detectadas**

5. MÉTRICAS DE PRUEBAS Y ASEGURAMIENTO DE CALIDAD

Se basan en el Catálogo de Pruebas de ÓrbiKa (63 casos de prueba), que cubre los 21 casos de uso con tres niveles por cada uno: prueba unitaria, prueba funcional y prueba de aceptación.

C1. Cobertura de casos de uso con plan de pruebas

Elemento	Descripción
Definición	Proporción de los 21 CU especificados que cuentan con al menos un caso de prueba diseñado en el catálogo.
Justificación	Para un sistema transaccional (compras, stock, cancelaciones) donde un error de lógica tiene impacto de negocio directo, planificar la prueba antes de construir reduce el riesgo de defectos costosos de corregir después.
Cómo se mide	Conteo de valores únicos de la columna «Funcionalidad / Característica» del Catálogo de Pruebas (63 filas, 21 valores únicos, uno por CU).
Resultado	**21 de 21 CU con plan de prueba = 100%**

C2. Automatización de pruebas planificadas

Elemento	Descripción
Definición	Proporción de los 63 casos de prueba del catálogo que están marcados como de modalidad «Automatizada» frente a «Manual».
Justificación	Las pruebas automatizadas son las que realmente pueden volver a ejecutarse en cada cambio sin costo adicional de tiempo; un equipo de 2 personas depende de ellas para no perder cobertura al avanzar con los 10 CU restantes.
Cómo se mide	Conteo de la columna «Modalidad» del Catálogo de Pruebas: las 21 pruebas unitarias están marcadas como automatizadas; las 21 funcionales y 21 de aceptación, como manuales.
Resultado	**21 de 63 automatizadas = 33.3% · 42 de 63 manuales = 66.7%**

C3. Ejecución de pruebas y densidad de defectos

Elemento	Descripción
Definición	Proporción de los 63 casos de prueba del catálogo que ya se ejecutaron contra el sistema real, y cantidad de defectos encontrados por cada mil líneas de código (KLOC).
Justificación	Es la métrica que normalmente demuestra si el software construido realmente cumple lo especificado; sin ejecución registrada, el catálogo es un plan de calidad, no todavía evidencia de calidad.
Cómo se mide	El Catálogo de Pruebas no contiene columnas de resultado de ejecución (aprobado/fallido) ni existe un registro de incidencias o bugs del proyecto.
Resultado	**No aplica — no se ha iniciado la ejecución formal del catálogo ni existe un registro de defectos; se recomienda abrir esta ejecución en la siguiente fase, priorizando los 11 CU que ya tienen código (ver B2)**

6. MÉTRICAS DE ATRIBUTOS DE CALIDAD EN OPERACIÓN

El SAD define 8 escenarios de calidad con el modelo de Len Bass (Fuente del estímulo → Estímulo → Entorno → Artefacto → Respuesta → Medida de respuesta), cada uno con una meta cuantitativa. Las cuatro métricas siguientes contrastan esas metas contra evidencia real; como el sistema no está desplegado en un entorno productivo con tráfico real, la mayoría se reporta como No aplica, en vez de simular una cifra que no se ha medido.

D1. Rendimiento bajo carga concurrente

Elemento	Descripción
Definición	Latencia promedio de la API ante 100 solicitudes simultáneas de búsqueda y consulta de productos.
Justificación	ÓrbiKa es un marketplace de dos lados (compradores y vendedores); si el catálogo responde lento en horas pico, la plataforma pierde la confianza que necesita para generar transacciones.
Cómo se mide	Meta definida en el escenario «Rendimiento — Búsqueda Concurrente de Productos» del SAD (sección 4): latencia ≤3 s para el 95% de las solicitudes con 200 usuarios concurrentes. Requiere un entorno desplegado (Render + Supabase) y una herramienta de prueba de carga (p. ej. k6 o Apache JMeter) para medirse.
Resultado	**No aplica — el sistema no está desplegado en un entorno productivo; no existe tráfico real que medir**

D2. Disponibilidad del servicio (uptime)

Elemento	Descripción
Definición	Porcentaje de tiempo mensual en que el sistema responde correctamente a las solicitudes de los usuarios.
Justificación	Una plataforma de productos perecibles depende de ventanas de tiempo cortas (antes del vencimiento); una caída del servicio en ese momento representa una pérdida de oportunidad real para el vendedor.
Cómo se mide	Meta definida en el escenario «Disponibilidad» implícito en las restricciones del SAD: ≥99% mensual, medido por el monitoreo propio de la infraestructura (Render + Supabase) una vez desplegada.
Resultado	**No aplica — no existe aún un despliegue activo que monitorear**

D3. Usabilidad del registro sin capacitación previa

Elemento	Descripción
Definición	Tiempo que tarda una persona sin experiencia previa en completar el registro y tasa de finalización de esa tarea.
Justificación	El público objetivo de ÓrbiKa incluye vendedores informales (puestos de mercado) sin experiencia tecnológica previa; si el registro es complejo, la plataforma pierde exactamente al tipo de usuario que busca atraer.

Elemento	Descripción
Cómo se mide	Meta definida en el escenario «Usabilidad — Registro de Vendedor sin Capacitación Previa» del SAD: registro completado en ≤3 min, con tasa de finalización ≥90%. Se mide con una prueba de usuario moderada (5 a 8 participantes reales).
Resultado	**No aplica de forma rigurosa — no se han ejecutado pruebas de usuario; observación informal: el formulario implementado (CU-01) tiene 5 campos y un selector de rol, un diseño consistente con la meta, pendiente de validar con usuarios reales**

D4. Mantenibilidad y costo de cambio

Elemento	Descripción
Definición	Tiempo de desarrollador (horas-persona) necesario para incorporar un cambio de alcance acotado, como una nueva modalidad de entrega.
Justificación	Con 10 de los 21 CU aún por construir, la facilidad para extender el sistema sin romper lo ya construido determina si el proyecto puede completarse dentro del cronograma académico.
Cómo se mide	Meta definida en el escenario «Mantenibilidad — Incorporación de un Nuevo Método de Entrega» del SAD: cambio implementado en menos de 4 horas-persona. No se ha cronometrado un cambio real de este tipo sobre el código construido.
Resultado	**Sin medición cronometrada — la métrica B5 (100% de modularidad, 0 violaciones de capas) es un indicio favorable para esta meta, pero no la mide directamente**

7. HALLAZGO: ACTUALIZACIÓN DEL AVANCE DE CONSTRUCCIÓN

El Informe Ejecutivo del proyecto (sección 5, tabla de avance ponderado) reporta la fila «Desarrollo de Software» en 0%, con la justificación «No se ha iniciado la codificación de backend, frontend ni base de datos». Esa cifra corresponde al corte de ese informe, anterior a la construcción del código evaluado en la sección 4 de este documento.

Hallazgo: a la fecha de este informe, sí existe código real y verificado para 11 de los 21 casos de uso (52.4%, métrica B2), con arquitectura en capas completa (100%, métrica B5) y un tamaño de 1,354 líneas de código. La fila «Desarrollo de Software: 0%» del Informe Ejecutivo ya no refleja el estado real del proyecto y debería actualizarse con las cifras de la sección 4 de este informe en la siguiente revisión de ese documento.

Las demás filas de esa tabla —Levantamiento de Requerimientos (100%), Análisis y Diseño (100%), Documentación de Gestión (100%)— sí coinciden con las métricas A1–A4 de este informe y no requieren corrección. La fila «Pruebas y Aseguramiento de Calidad» (15%, «planificado») tampoco contradice la métrica C1 de este informe (100% de CU con plan de prueba, pero 0% ejecutado): ambas cifras miden cosas distintas y son compatibles entre sí.

8. CONCLUSIONES

¿Qué métricas resultaron más relevantes según el tipo de software?

ÓrbiKa es un marketplace transaccional de dos lados (compradores y vendedores) que todavía está en etapa de construcción (11 de 21 casos de uso con código, según la métrica B2). Para este tipo de software, en esta etapa, las métricas más relevantes fueron:

- **Cobertura y trazabilidad de requerimientos (A1–A3):** en un sistema con reglas de negocio sensibles al dinero y al stock (RN-10, cancelación con restitución de stock; la función transaccional crear_pedido_transaccion), construir sin especificación trazable es el riesgo más costoso de los cuatro grupos evaluados.
- **Cobertura de plan de pruebas por caso de uso (C1):** un marketplace depende de que las transacciones se completen correctamente; el 100% de cobertura del catálogo es más valioso ahora que, por ejemplo, la automatización (C2), porque primero hay que asegurar que todo flujo de negocio tenga al menos una prueba diseñada.
- **Cobertura de CU implementados (B2) y modularidad (B5):** miden si el proyecto avanza a un ritmo sostenible con solo dos integrantes, y si lo construido hasta ahora facilita o dificulta terminar los 10 CU restantes.

Las métricas de atributos de calidad en operación (sección 6: rendimiento, disponibilidad, usabilidad medida) resultaron las menos relevantes por ahora —todas N/A—, no porque no importen, sino porque requieren un sistema desplegado con usuarios reales. Para un marketplace, esas son exactamente las métricas que pasarán a ser las más relevantes en cuanto el sistema llegue a producción, porque de ellas depende que compradores y vendedores confíen en seguir usando la plataforma.